Enhancing Accessibility of a Web-based Scheme IDE to Improve Inclusion in Computing

Jacob Bell
belljake@grinnell.edu
Grinnell College
Grinnell, IA, USA

Nicole Moreno González
morenoni@grinnell.edu
Grinnell College
Grinnell, IA, USA

Charles Wade
wadechar@grinnell.edu
Grinnell College
Grinnell, IA, USA

1 Introduction

Accessibility, in many contexts, is a legal requirement for equitable engagement in education [14]. For blind and low-vision (BLV) and neurodivergent (ND) students, inaccessible software tools can undermine learning and confidence. Although accessibility in higher education technology has improved, specialized tools such as Integrated Development Environments (IDEs) often lag behind, limiting scaffolding for beginners.

Prior research emphasizes accessibility in development, yet few studies consider the full software engineering process [12]. While languages like Quorum [1] are designed with accessibility in mind, most languages and IDEs are retrofitted inadequately. Scheme-based IDEs, though less common, remain central in some curricula for their "functional first" approach and in K–12 programs like Bootstrap [13]. However, a review of Scheme IDEs revealed poor screen reader support and limited features for BLV and ND users. Scamper, an open-source web Scheme IDE [10], reflects this gap: despite pedagogical strengths, it poses significant accessibility challenges.

Through a manual WCAG 2.1 [4] audit and screen reader testing, we identified non-semantic HTML, keyboard-inaccessible elements, and no tools for navigating nested code. These barriers prevent BLV and ND learners from fully benefiting from Scamper. We addressed them through targeted fixes and new features aligned with Universal Design principles, improving usability for all students.

2 Background and Related Work

Mountapmbeme et. al. [9] claims that a sighted programmer can quickly navigate information without moving their cursor, but this is not possible for a BLV programmer dependent on a screen reader. They also suggested that the techniques that one might use to support BLV users may differ from those appropriate for sighted users. Fortunately, there have already been efforts to improve accessibility in IDEs that address the difficulties that BLV users face, particularly when navigating large code bases. Some notable examples are StructJumper [2] and Aural Tree Navigator [15], two Eclipse plugins that build a hierarchical tree view based on the structural nesting of a Java class. In a user study with blind programmers, participants were able to complete navigation tasks more quickly with StructJumper, and non-sighted users of Aural Tree Navigator could complete programming tasks in a complicated, multi-file project. These examples demonstrate the value of structural overviews for non-visual navigation.

Research on supporting ND students also highlights the need for flexible, multimodal learning tools. For example, in reviewing appropriate accommodations, Brown University's Sheridan Center for Teaching and Learning notes that "Some students need to be able to go back and look at directions several times to make sure they've understood," and that others "report that using screen-readers, as opposed to just reading text, enables them to learn better" [7]. These findings suggest that screen reader compatibility and navigable content design enhance usability for many learners.

3 Approach and Uniqueness

3.1 Preliminary Assessment of the Scamper IDE

Drawing upon issues identified in the literature, we conducted a manual inspection of the Scamper IDE, revealing four main accessibility issues:

Hover-dependent functionality: Scamper included features that are only visible on hover. This design choice poses a significant accessibility barrier for users who rely exclusively on keyboards and screen readers [16]. Since hover states are not keyboard-reachable, the information or controls they reveal remain hidden from assistive technologies.

Lack of semantic labeling: As Figure 1 suggests, interactive elements in the editor pane, lack meaningful labels or ARIA attributes. As a result, screen readers fail to convey the purpose or presence of these controls.

Barriers to readable code for collaboration: The lack of accessible indentation feedback makes it difficult to follow conventional formatting practices. This can lead to code that is syntactically correct but visually disorganized, potentially reducing readability for sighted collaborators and ND.

Limited code navigation support: Scamper's editor does not support structured navigation of Scheme code, particularly its parenthetical nesting. Screen readers are therefore unable to convey the syntactic hierarchy effectively, making it difficult for all users, but particularly BLV, to locate specific expressions larger codebases.

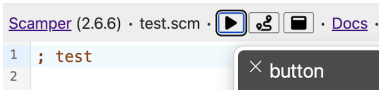


Figure 1: The screen reader is unable to announce the run button meaningfully. It just reads 'button' when focusing it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
TAPIA '26, Atlanta, GA, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/02
https://doi.org/XXXXXXX.XXXXXXX

3.2 Addressing Scamper's Deficiencies

Replacing hover-dependent features: We removed all hover-dependent interactions and replaced them with always-visible features. These controls now remain reachable via keyboard and interpretable by screen readers.

Introducing semantic labeling: We assigned appropriate attributes to all relevant interactive elements. This included adding ARIA-label and role attributes to buttons, toggles, and navigation links. With these improvements, screen readers can now identify and navigate interactive elements like buttons.

Facilitating readable code through autoindentation: We introduced an autoindentation feature triggered by keyboard shortcuts. This feature reformats code according to standard indentation conventions without disrupting the user's cursor position, allowing learners to maintain focus while ensuring their code remains readable to sighted peers and instructors.

Enhancing structural awareness with an interactive abstract syntax tree: As Ehtesham-Ul-Haque et al. [6] suggest, "There is growing evidence that BLV users find tabular structure more useful than raw data". Hence, we introduced an explicit view of the abstract syntax tree (AST), which parses written code into a displayed tree organized by nesting levels that are also accessible by screen readers and can be triggered to jump to that code segment.



Figure 2: The Screen Reader announces the run button as 'run, button' when focusing it.

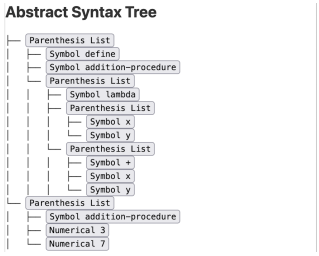


Figure 3: The parse tree for a simple Scheme program. Each parenthetical expression is represented as a node, where each item between the parentheses is a child of that node.

4 Results and Contributions

One of the lessons we learned was that Scamper's development highlights the cost of overlooking accessibility in early design. Our work demonstrated that retrofitting accessibility is far more difficult than building it in from the start, as prior research has shown [11, 17]. Beyond implementation, testing Scamper revealed the importance of using assistive technologies and treating accessibility as an ongoing process as technologies evolve.

Features designed for BLV and ND users also allowed us to acknowledge that accessibility enhancements can benefit all learners.

Looking forward, Scamper aims to expand its accessibility support beyond BLV and ND users. We hope to make Scamper more accessible to a wider range of disabilities and address challenges related to sound, images, and other media supported by the IDE. Direct collaboration with students and faculty, especially those with disabilities, will guide solutions that reflect real user needs [3, 5, 8]. In summary, meaningful accessibility work requires faculty and students to commit to inclusive design shaped by active engagement with the communities it intends to serve.

References

[1] AccessComputing. [n. d.]. Quorum Programming Language. https://www.washington.edu/accesscomputing/quorum-programming-language-0

[2] Catherine M. Baker, Lauren R. Milne, and Richard E. Ladner. 2015. StructJumper: A Tool to Help Blind Programmers Navigate and Understand the Structure of Code. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. Association for Computing Machinery, New York, NY, USA, 3043–3052. doi:10.1145/2702123.2702589

[3] Cynthia L. Bennett, Erin Brady, and Stacy M. Branham. 2018. Interdependence as a Frame for Assistive Technology Research and Design. In *Proceedings of the 20th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '18)*. Association for Computing Machinery, New York, NY, USA, 161–173. doi:10.1145/3234695.3236348

[4] The World Wide Web Consortium. [n. d.]. Web Content Accessibility Guidelines. https://www.w3.org/TR/2008/REC-WCAG20-20081211/

[5] Tim Coughlan, Thomas Daniel Ullmann, and Kate Lister. 2017. Understanding Accessibility as a Process through the Analysis of Feedback from Disabled Students. In *Proceedings of the 14th International Web for All Conference (W4A '17)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3058555.3058561

[6] Md Ehtesham-Ul-Haque, Syed Mostofa Monsur, and Syed Masum Billah. 2022. Grid-Coding: An Accessible, Efficient, and Structured Coding Paradigm for Blind and Low-Vision Programmers. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*. ACM, Bend OR USA, 1–21. doi:10.1145/3526113.3545620

[7] The Harriet W. Sheridan Center for Teaching and Learning. [n. d.]. Supporting Neurodivergent Students in the Classroom | Sheridan Center for Teaching and Learning | Brown University. https://sheridan.brown.edu/resources/inclusive-teaching/supporting-neurodivergent-students-classroom

[8] Hyung Nam Kim. 2024. Guidance for Best Practices in Participatory Design Involving People with Visual Impairment. *Ergonomics in Design* 32, 2 (April 2024), 48–54. doi:10.1177/10648046211058248

[9] Aboubakar Mountapmbeme, Obianuju Okafor, and Stephanie Ludi. 2022. Addressing Accessibility Barriers in Programming for People with Visual Impairments: A Literature Review. *ACM Trans. Access. Comput.* 15, 1 (March 2022), 7:1–7:26. doi:10.1145/3507469

[10] Peter-Michael Osera. [n. d.]. Scamper: A mini-Scheme for teaching. https://scamper.cs.grinnell.edu

[11] Débora Maria Barroso Paiva, André Pimenta Freire, and Renata Pontin de Mattos Fortes. 2021. Accessibility and Software Engineering Processes: A Systematic Literature Review. *Journal of Systems and Software* 171 (Jan. 2021), 110819. doi:10.1016/j.jss.2020.110819

[12] Nataša Rajh, Klaus Miesenberger, and Reinhard Koutny. 2024. Accessibility in the Software Engineering (SE) Process and in Integrated Development Environments (IDEs): A Systematic Literature Review. In *Computers Helping People with Special Needs*, Klaus Miesenberger, Petr Peňáz, and Makoto Kobayashi (Eds.). Springer Nature Switzerland, Cham, 11–18. doi:10.1007/978-3-031-62846-7_2

[13] Emmanuel Schanzer, Kathi Fisler, and Shriram Krishnamurthi. 2018. Assessing Bootstrap: Algebra Students on Scaffolded and Unscaffolded Word Problems. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education* (Baltimore, Maryland, USA) *(SIGCSE '18)*. Association for Computing Machinery, New York, NY, USA, 8–13. doi:10.1145/3159450.3159498

[14] Natalie L. Shaheen. 2022. Technology accessibility: How U.S. K-12 schools are enacting policy and addressing the equity imperative. *Computers Education* 179 (April 2022). doi:10.1016/j.compedu.2021.104414

[15] Ann C. Smith, Justin S. Cook, Joan M. Francioni, Asif Hossain, Mohd Anwar, and M. Fayezur Rahman. 2003. Nonvisual tool for navigating hierarchical structures. *SIGACCESS Access. Comput.* 77–78 (Sept. 2003), 133–139. doi:10.1145/1029014.1028654

[16] Princeton University. [n. d.]. Write Accessible Code. https://digital.accessibility.princeton.edu/how/develop

[17] Brian Wentz, Paul T. Jaeger, and Jonathan Lazar. 2011. Retrofitting accessibility: The legal inequality of after-the-fact online access for persons with disabilities in the United States. *First Monday* 17, 11 (Nov. 2011). doi:10.5210/fm.v16i11.3666